

C++ OOP 기말 프로젝트 보고서

주제: 텍스트 RPG 전투 시뮬레이터

학번:2020575068, 이름: 최원욱

1. 프로그램 개요

1-1. 주제 선택 이유

텍스트 RPG 를 선택한 이유는 플레이어와 몬스터는 공통적으로 체력, 공격력, 방어력과

같은 정보를 가지지만, 직업이나 몬스터 종류에 따라 공격 방식과 스킬이 달라진다.

이러한 구조는 상속과 다형성을 자연스럽게 적용할 수 있어

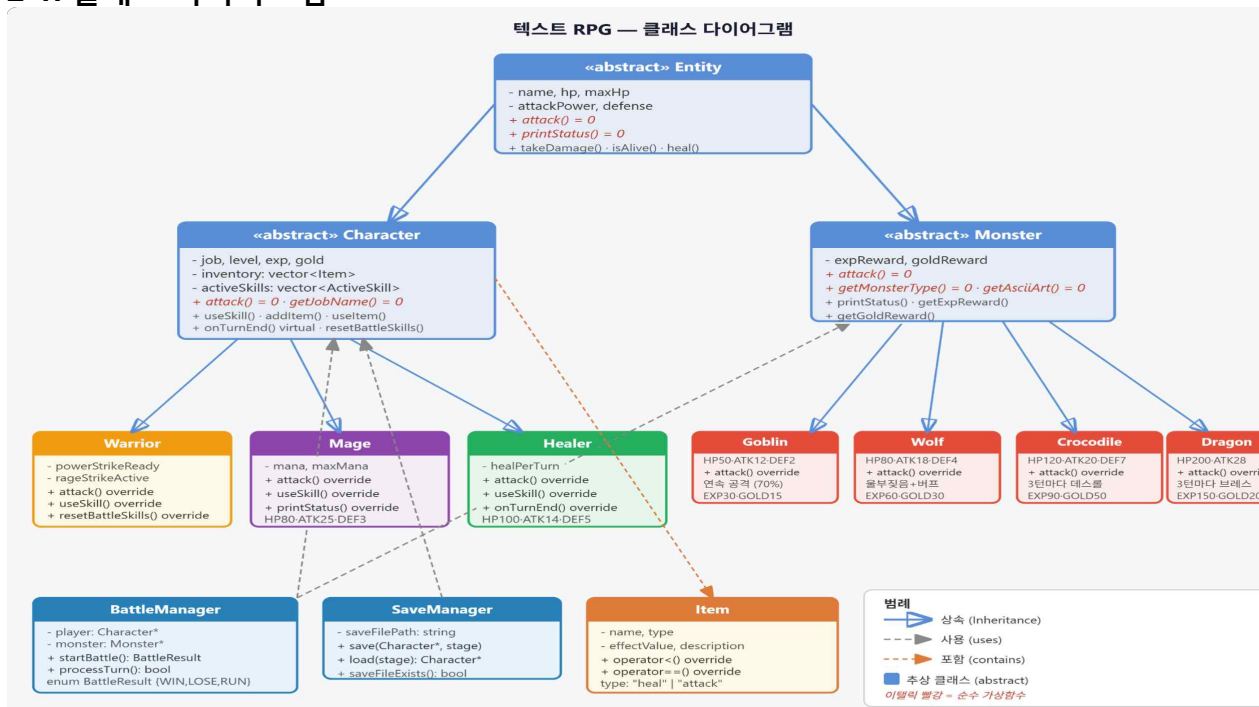
객체지향 설계의 장점을 잘 활용할 수 있다고 생각하여 프로젝트 주제로 선정하였다.

1-2. 구현한 기능 목록

- 캐릭터 생성: 이름 입력 및 직업 선택 (전사 / 마법사 / 힐러)
- 몬스터 4종: 고블린 → 늑대 → 악어 → 드래곤 (스테이지별 능력치 설계 + 고유 공격 패턴)
- 턴제 전투: 공격 / 아이템 사용 / 도망 / 선택
- 아이템 시스템: 상점에서 HP 포션·공격 포션 구매 → 인벤토리 보관 → 전투 중 사용
- 전투 결과 처리: 승리(경험치·골드 획득) / 패배(게임 오버) / 도망(50% 확률)
- 저장·불러오기: savefile.txt에 전체 게임 상태 저장, 프로그램 재시작 후 복원
- 레벨업: 경험치 축적 → 능력치 선택 강화 → Lv.2 스킬 해제

2. 클래스 설계

2-1. 클래스 다이어그램



2-2. 클래스 책임 분리 근거

클래스	역할	설계 근거
Entity	HP·ATK·DEF 공통 속성, takeDamage(), isAlive()	Character와 Monster의 중복 코드 제거. 단일 책임 원칙(SRP).
Character	인벤토리, 골드, 경험치, 레벨업	Entity 상속 후 플레이어 전용 기능만 추가.
Monster	보상(EXP·골드), ASCII 아트, 공격 패턴	새 몹스터 추가 시 기존 코드 무수정.
BattleManager	턴제 전투 진행, 애니메이션, 로그	Character/Monster의 구체 타입을 모르므로 동적 할당.
SaveManager	파일 I/O 전달: save(), load()	파일 형식 변경 시 이 클래스만 수정.
Item	아이템 데이터 (이름·타입·효과값·설명)	데이터 클래스로 분리. Character가 직접 내부를 알 필요 없음.

3. 핵심 개념 적용

3-1. 다형성

Entity에 순수 가상 함수를 선언하여 Character와 Monster의 공통 인터페이스를 정의하였다.

```
virtual int attack() = 0;
virtual void printStatus() const = 0;
```

BattleManager는 Character*와 Monster* 포인터만 보유하며, player->attack()을 호출하면 런타임에 전사·마법사·힐러 각각의 override된 메서드가 자동으로 호출된다.

또한 턴 종료 훈(onTurnEnd)을 Character에 virtual로 선언하여, dynamic_cast 없이 Healer만 자동 회복이 동작하도록 순수한 다형성으로 처리하였다.

```
virtual void onTurnEnd() {} // 턴 종료 후 (Healer 자동 회복 등)
```

(character.h, 61줄)

```
void onTurnEnd() override;
```

(Jobs.h, 64줄)

```
player->onTurnEnd();
```

(BattleManager.cpp, 227~228줄)

3-2. 동적 할당

직업 선택은 컴파일 타임이 아닌 런타임에 결정되므로, 동적 할당으로 적절한 파생 클래스를 생성하였다.

```
Character* player = nullptr;
if (choice == 1) player = new Warrior(name);
else if (choice == 2) player = new Mage(name);
else player = new Healer(name);
```

(main.cpp, 253~256줄)

SaveManager::load()에서도 저장된 직업 문자열을 읽어 적절한 파생 클래스를 new로 생성하여 반환한다. 사용 후 delete로 명시적으로 해제한다

```
// 직업에 따라 파생 클래스 생성
Character* player = nullptr;
if (job == "전사") {
    player = new Warrior(name);
} else if (job == "마법사") {
    player = new Mage(name);
} else if (job == "힐러") {
    player = new Healer(name);
} else {
    throw std::runtime_error("알 수 없는 직업 데이터: " + job);
}
```

(SaveManager.cpp 63~73줄)

3-3. 예외 처리

5가지 예외 상황에서 throw를 사용하고, 호출 측에서 try-catch로 안전하게 처리하였다.

throw 위치	예외 종류	catch 위치
SaveManager::load() - 파일 없음	runtime_error	main.cpp 메인 루프
SaveManager::save() - 파일 열기 실패	runtime_error	gameLoop() 내부
Character::useItem() - 아이템 없음	runtime_error	BattleManager::processTurn()
getPlayerChoice() - 잘못된 입력	invalid_argument	startBattle() while 루프
getPlayerChoice() - 범위 초과	out_of_range	startBattle() while 루프

```
// 예외 처리 ㉔ - 파일 열기 실패
if (!file.is_open()) {
    throw std::runtime_error("저장 파일을 열 수 없습니다: " + saveFilePath);
}
```

```
// 예외 처리 ㉕ - 저장 파일 없음
if (!file.is_open()) {
    throw std::runtime_error("저장 파일이 없습니다. 새 게임을 시작해주세요.");
}
```

(SaveManager.cpp , 18~21줄 , 51~53줄)

```
while (choice == -1) {
    try {
        choice = getPlayerChoice();
    } catch (const std::exception& e) {
        std::cout << Color::RED << "[오류] " << e.what() << Color::RESET << "\n";
    }
}
```

(battlemanager.cpp , 199~203줄)

4. 선택 기술 요건 적용

적용 기술	코드 위치	적용 내용
연산자 오버로딩	Item.cpp - operator<, operator==	STL sort/find_if와 연동하기 위해 비교 연산자를 오버로딩. 인벤토리 정렬 및 아이템 검색에 활용.
람다 표현식	Character.cpp - findItem(), sortInventory()	std::find_if와 std::sort에 람다를 전달하여 별도 함수 선언 없이 인벤토리 조회·정렬을 간결하게 구현.
namespace	BattleManager.h - Color namespace	ANSI 색상 코드를 Color 네임스페이스로 묶어 전역 오염을 방지하고, Color::RED인지 명시적으로 사용.
constexpr	BattleManager.h - Color 상수 전체	컴파일 타임 상수로 선언하여 런타임 오버헤드 없이 색상 코드를 사용.

```
// 이름 기준 정렬 (STL sort에서 사용)
bool Item::operator<(const Item& other) const {
    return name < other.name;
}

// 이름 기준 동등 비교 (STL find에서 사용)
bool Item::operator==(const Item& other) const {
    return name == other.name;
}
```

(Item.cpp , 11~19줄)

```
Item* Character::findItem(const std::string& itemName) {
    auto it = std::find_if(inventory.begin(), inventory.end(),
        [&itemName](const Item& i) { return i.getName() == itemName; });
    return (it != inventory.end()) ? &(*it) : nullptr;
}
```

```
void Character::sortInventory() {
    std::sort(inventory.begin(), inventory.end(),
        [](const Item& a, const Item& b) {
            if (a.getType() != b.getType()) return a.getType() > b.getType();
            return a.getName() < b.getName();
        });
}
```

(Character.cpp, 55~58줄, 61~67줄)

5. 테스트 목록

main.cpp와 별개의 test.cpp로 작성하여 독립적으로 빌드·실행한다.

테스트 케이스	분류	검증 내용	결과
Warrior 초기 HP == 120	정상	생성자 HP 설정 검증	PASS
Mage 초기 HP == 80	정상	직업별 초기 스탯 검증	PASS
Healer 초기 HP == 100	정상	직업별 초기 스탯 검증	PASS
takeDamage 정상 계산 (18-8=10)	정상	데미지 - 방어력 공식 검증	PASS
takeDamage(0) → HP 변화 없음	경계값	0 데미지 시 HP 불변 검증	PASS
takeDamage(방어력 이하) → 최소 1	경계값	최소 1 데미지 보장 검증	PASS
치명 데미지 후 HP >= 0	경계값	HP 음수화 방지 검증	PASS
isAlive() → 살아있을 때 true	정상	HP > 0 시 생존 상태 검증	PASS
isAlive() → 사망 후 false	실패 케이스	치명 데미지 후 사망 전환 검증	PASS
Warrior 초기 골드 == 0	정상	게임 시작 시 골드 0 초기화 검증	PASS

10개 테스트 모두 PASS. 정상 케이스 5개, 경계값 3개, 실패 케이스 1개 포함.

```

===== 단위 테스트 시작 =====

[PASS] Warrior 초기 HP == 120
[PASS] Mage 초기 HP == 80
[PASS] Healer 초기 HP == 100
[PASS] takeDamage 정상 계산 (18-8=10)
[PASS] takeDamage(0) → HP 변화 없음 [경계]
[PASS] takeDamage(방어력 이하) → 최소 1 데미지 [경계]
[PASS] 치명 데미지 후 HP >= 0 [경계]
[PASS] isAlive() → 살아있을 때 true
[PASS] isAlive() → 사망 후 false [실패 케이스]
[PASS] Warrior 초기 골드 == 0

===== 10 PASS / 0 FAIL =====

```

6. 한계 및 개선 방향

6-0. 추가 기능

Windows MCI API로 bgm.mp3루프 재생 (Pixabay- Under The Elven Star - Fantasy RPG Music , kaazoom)

ANSI 컬러 + 콘솔 UI (체력 비율에 따라 HP바 색이 초록→노랑→빨강으로 변함)

ASCII 아트 (몬스터 4종(고블린/늑대/악어/드래곤)도 각자 아트 보유.)

직업별 고유 스킬

전사: 분노의 일격 (다음 공격 2배),

마법사: 마력 폭발 (마나 전소 고데미지),

힐러: 신성 치유 (HP 40% 즉시 회복)

— 모두 Lv.2 달성 시 해금되고 전투당 사용횟수 제한.

전투 크리티컬 시스템

15% 확률 크리티컬 1.5배 데미지

6-1. 미구현 기능

- 다중 저장 슬롯: 현재 단일 savefile.txt만 지원한다. 슬롯 번호 추가 시 여러 세이브를 관리할 수 있다.
- 몬스터 드롭 아이템: 전투 승리 후 아이템을 획득하는 기능이 없다.
드롭 시스템을 추가하면 아이템 수급 경로가 다양해진다.

6-2. 코드 리팩토링 후보

- printStatus 중복 제거: Mage와 Healer의 printStatus()가 기반 클래스와 중복되는 코드가 많다. 공통 부분을 추출하면 유지보수가 쉬워진다.

6-3. AI 도구 활용 명시

프로젝트 개발 과정에서 Claude AI를 활용하였다. 클래스 구조와 UML 설계, ANSI 색상 출력, Windows MCI 오디오 API 연동 등의 구현 방법을 참고하였으며, 제안된 내용을 그대로 사용하지 않고 프로젝트에 맞게 수정·보완하여 적용하였다. 게임 규칙, 직업별 스킬, 밸런스, 기능 구성 등은 직접 검토하고 결정하였다.

ASCII Art Archive

<https://ascii.co.uk/art> - jgs— 전사, SSt— 마법사 , ZOT— 힐러

Pixabay - Under The Elven Star - Fantasy RPG Music , kaazoom

(출처)